TEACHING SERIES * INTRO TO AI SYSTEMS

Retrieval-Augmented Generation (RAG)

Focus: Chunking Strategies - with examples & applications

For builders learning how to ground LLMs in real documents

What we'll cover

A practical path from RAG basics -> chunking craft

01	What RAG is and why it exists
02	Where chunking sits in the pipeline
03	Six chunking strategies with worked examples
04	How to choose a strategy for your data
05	Real applications + common failure modes

Search your knowledge base for passages related to the u

Insert those passages into the LLM prompt as grounded co

The model answers using that context - not only its training mer

Hallucinations

Models invent confident-sounding facts when knowledge is missing.

Stale knowledge

Training cutoffs miss policies, prices, and last week's docs.

Private data

Company wikis, tickets, and PDFs are never in the base model.

Citations

Users need sources - RAG returns passages you can show.

Documents

PDFs, HTML, MD

Chunk

Split into units

Embed

Vectors + metadata

Store

Vector DB

Retrieve

Top-k for query

Answer

LLM + context

Teaching focus: Chunk is the design decision that makes or breaks retrieval.

- Retrieves irrelevant text
- Burns context tokens
- Harder to focus

- Loses context
- Needs more chunks
- Noisy ranking

es

01
Fixed-size
Char / token windows

02
Overlap
Sliding window

03
Recursive
Headings & structure

04
Semantic
Meaning shifts

05
Parent-child
Search small, read large

06
Proposition
Atomic facts

01 * Fixed-size chunking

How it works

- Choose size: e.g. 500-1000 tokens
- Walk through document sequentially
- Break on whitespace near the limit
- Store chunk_id, source, offsets as metadata

Best for

- Homogeneous text
- Quick prototypes
- Weak structure

Example * Fixed-size on a policy doc

Size ~80 tokens, no overlap

SOURCE PARAGRAPH

Employees may work remotely up to three days per week. Managers must approve schedules by Friday. Equipment stipends are $500 per year. Security training is mandatory before access is granted.

Chunk A

Employees may work remotely... Managers must approve schedules by Friday.

Chunk B

Equipment stipends are $500 per year. Security training is mandatory...

02 * Overlapping / sliding window

Why overlap?

- Questions span two adjacent chunks
- Overlap preserves bridging sentences
- Typical overlap: 10-20%
- Tradeoff: more storage + duplicates

Visual

Chunk 1 ..... [overlap]

[overlap] ..... Chunk 2

Example * Overlap saves a multi-hop fact

Query: "Stipend + who approves remote days?"

Without overlap, stipend and approval may land in different chunks.

Chunk 1 (with overlap)

...Managers must approve... Equipment stipends are $500 per year...

- Approval + stipend co-occur
- Single retrieval answers full question
- Fewer "only found half" failures

03 * Recursive / structure-aware

- 1. Split on headings / markdown #
- 2. If too big -> split on paragraphs
- 3. Then sentences
- 4. Last resort -> characters

Best for

- Docs with headings
- Markdown / Confluence
- API references
- Handbooks

Example * Recursive keeps sections intact

Naive fixed cut

Refunds
...
Shipping
Orders ship within... [cut]

Recursive

Chunk 1: ## Refunds + body
Chunk 2: ## Shipping + body

04 * Semantic chunking

Split when meaning shifts - not when counter hits N

Embed sentences. When similarity drops below threshold, start new chunk.

Idea	Cost	Win	Watch
Topic coherence	Extra embedding passes	Mixed / narrative docs	Threshold tuning

Example * Semantic split on a blog post

Chunk A * Product launch

We shipped v2.0 with offline sync...

Chunk B * Pricing change

Starting May 1, Pro is $29/mo...

Chunk C * Hiring

We're hiring two backend engineers...

05 * Parent-child (small-to-big)

- Child: small for accurate search
- Parent: section for rich LLM context
- On hit: fetch parent_id -> inject parent
- Also: hierarchical / multi-granularity RAG

PARENT: Section 3 Security Policy

Child 3.1 MFA required

Child 3.2 VPN only

Child 3.3 Device wipe

06 * Proposition / atomic chunking

Rewrite text into standalone facts, then embed each

Use LLM to extract self-contained facts (no "it / this"). Great for dense reports.

Original

Acme Plan covers dental after 6-month wait and vision after 90 days for full-time staff.

Propositions

- Dental after 6 months
- Vision after 90 days
- Full-time staff only

- Noisy embedding
- Too much context for LLM

- "It expires" - what is it?
- Missing exceptions

Fix: search with small children * answer with full parent section

Parent-child * Worked example

PARENT (not embedded)

Section 3: 3 days remote * approval by Friday * $500 stipend * security training

Child	Embedded text	parent_id
C1	Remote work up to 3 days/week	S3
C2	Manager approval by Friday	S3
C3	Equipment stipend $500/year	S3
C4	Security training before VPN	S3

Parent-child * Retrieval flow

Query: "How much is the equipment stipend?"

1	**User question**	How much is the equipment stipend?
2	**Vector search**	Matches child C3
3	**Read parent_id**	C3 -> parent Section 3
4	**Fetch parent**	Load full section into prompt
5	**LLM answer**	"$500 per year"

Parent-child * What gets embedded?

Embed children only * store parents as linked text

Piece	Embedded?	Stored where	Used for
Child (sentence)	Yes	Vector DB	Search
Parent (section)	No	DB / doc store	LLM context

Why not embed parents? Duplicate results, noisier search, unclear LLM input

Index time

- Save parent as text + parent_id
- Embed children with parent_id
- Parent not in vector index

Query time

- Search children only
- Hit C3 -> fetch parent
- Send parent to LLM

Proposition * Why it exists

Multiple facts and vague pronouns in one embedding

Fix: one standalone fact per chunk

- Several facts in one paragraph
- Pronouns ("it") embed poorly
- User gets 3 unrelated facts

X It covers dental after 6 months.

Y Acme Plan covers dental after 6-month wait.

X Vision is after 90 days for FT staff.

Y Acme Plan covers vision after 90 days.

Proposition * Worked example

Query: "When does vision coverage start?"

Original paragraph

P1: Dental after 6-month wait

P2: Vision after 90-day wait

P3: Full-time status required

Retrieval returns P2 only

- Not P1 or P3 - less noise
- Answer: "90 days"
- Fixed chunking returns whole paragraph

Strategy comparison at a glance

Pick for quality first - optimize cost second

Strategy	Precision	Context	Cost	Best fit
Fixed-size	Medium	Variable	Low	Prototypes
Overlap	Higher	Better	Low+	Policies
Recursive	High	High	Low	Structured docs
Semantic	High	High	Med	Mixed topics
Parent-child	Very high	Very high	Med	Long manuals
Proposition	Very high	Low-expand	High	Factoid Q&A

How to choose a strategy

Start simple; upgrade when retrieval evals fail

Most docs have headings?	-> Recursive (+ overlap)
Long chapters, precise Qs?	-> Parent-child
Mixed topics in one file?	-> Semantic
Need exact factual answers?	-> Proposition
Shipping MVP this week?	-> Fixed-size + 10-15% overlap

Customer support

Recursive + overlap

Cite article section

Legal / compliance

Parent-child

Clause hits + context

Codebase assistant

Function / class chunks

Don't split mid-function

Research / papers

Semantic by section

Methods vs results

HR policies

Recursive + proposition

Benefits factoid Qs

Internal search

Fixed + metadata

Hybrid keyword + vector

Worked application * Employee handbook Q&A

One corpus, three query types

Query type	Example	Chunk strategy
Policy lookup	"How many remote days?"	Recursive by section
Multi-fact	"Stipend + approval?"	Overlap or parent-child
Benefits factoid	"Vision wait period?"	Proposition chunks

Common pitfalls

Ignoring metadata

Store source, heading, page, timestamps.

One size for all

PDFs != chat logs != code.

No evaluation set

Build 20-50 questions; measure recall@k.

Wrong chunking time

Index-time chunks must match query-time.

Orphan pronouns

"It expires in 30 days" without what "it" is.

Over-chunking tables

Keep tables together or summarize.

Teaching checklist * best practices

Use this as a lab rubric

Y	Match chunk size to embedding model (256-1024 tokens)
Y	Prefer structure-aware splits over blind cuts
Y	Add 10-20% overlap when answers span boundaries
Y	Attach rich metadata for filters and citations
Y	Evaluate with real user questions before tuning
Y	Re-chunk when document structure changes
Y	Log retrieved chunks during demos

Classroom lab (30-45 min)

Hands-on: same corpus, competing chunkers

1. Pick a short handbook or README (~5-10 pages)
2. Build three indexes: fixed, recursive, parent-child
3. Write 10 questions (5 easy, 3 multi-hop, 2 adversarial)
4. Score recall@3 and answer correctness
5. Discuss: which strategy won - and why?

RAG grounds LLMs

Retrieve -> Augment -> Generate

Chunking is a product decision

Not just preprocessing

Structure beats blind cuts

Headings, parents, propositions

Measure, then upgrade

Eval questions before fancy chunkers

NEXT UP

Embeddings, retrieval, and evaluation

Questions? Try the lab - then we'll score your chunkers.